



Instituto Tecnológico de Costa Rica

Escuela de Ingeniería en Computadores

Curso: CE-1101 Introducción a la programación

Profesor: Elloth Alberto Ramírez Trejos

DOCUMENTO TÉCNICO

Proyecto: Imaginary Battles

Autor: Francisco Ramírez Delgado

Carné: 2025112972

Fecha: 9 de Mayo 2026

Índice

Palabras clave y Acrónimos	3
Introducción	4
Marco Teórico	5
Descripción del Programa	6
Arquitectura de módulos	6
Diagrama de flujo general del sistema	6
Resultados Obtenidos	7
Descripción de las Funciones Principales	7
Descripción de la Interfaz Gráfica	8
Conclusiones y Recomendaciones	9
Conclusiones	9
Recomendaciones	9
Bibliografía	10

Palabras clave y Acrónimos

Palabras clave: batallas por turnos, interfaz gráfica, programación orientada a objetos, Python, recursividad, captura de personajes

Acrónimos:

- **HP:** Hit Points — Puntos de vida de un personaje en combate.
- **ATK:** Attack — Estadística de ataque que determina el daño causado.
- **DEF:** Defense — Estadística de defensa que reduce el daño recibido.
- **UI:** User Interface — Interfaz visual con la que interactúa el jugador.
- **IA:** Inteligencia Artificial — Lógica automática que controla las decisiones del Hollow.
- **OOP:** Object-Oriented Programming — Paradigma de programación orientado a objetos.
- **TEC:** Tecnológico de Costa Rica.

Introducción

El presente documento técnico describe el diseño, la arquitectura y el funcionamiento del videojuego Imaginary Battles, desarrollado como proyecto final del curso CE-1101 Introducción a la Programación del Instituto Tecnológico de Costa Rica.

Imaginary Battles es un juego de batallas por turnos implementado en Python con interfaz gráfica desarrollada mediante la biblioteca Tkinter. El jugador asume el rol de Guardián de las Historias, cuya misión es recorrer cinco tierras del Imaginario para derrotar a los Huecos (Hollows) que las amenazan y restaurar los Fragmentos de Memoria perdidos.

Cada batalla enfrenta al jugador contra un Hollow controlado por una inteligencia artificial simple que decide aleatoriamente entre atacar o cambiar de personaje. El sistema de combate se basa en estadísticas individuales de cada personaje: puntos de vida (HP), poder de ataque (ATK) y defensa (DEF). Cuando un personaje es derrotado, el vencedor lo captura y lo incorpora a su equipo, lo que añade una capa estratégica al desarrollo del juego.

El objetivo de este documento es presentar de forma ordenada las decisiones técnicas tomadas durante el desarrollo, describir las funciones más relevantes del sistema y exponer los resultados obtenidos.

Marco Teórico

El sistema fue desarrollado en su totalidad utilizando el lenguaje de programación Python 3, seleccionado por su claridad sintáctica, su soporte nativo para programación orientada a objetos y su amplio ecosistema de bibliotecas.

Tecnologías y conceptos empleados:

- **Python 3.x:** Lenguaje principal del proyecto. Se utilizaron módulos estándar como random (para la IA del Hollow) y los sistemas de archivos de texto para la carga de personajes.
- **Tkinter:** Biblioteca estándar de Python para el desarrollo de interfaces gráficas. Permite crear ventanas, botones, etiquetas, barras de progreso y widgets personalizados mediante Canvas. Toda la interfaz visual del juego fue construida con Tkinter.
- **Pillow (PIL):** Biblioteca de procesamiento de imágenes utilizada para cargar, redimensionar y convertir las imágenes PNG de los personajes al formato PhotoImage que Tkinter puede renderizar.
- **Programación orientada a objetos (OOP):** El diseño del juego se apoya en dos clases principales: Personaje, que encapsula los atributos y comportamientos de cada combatiente, y Entrenador, que agrupa el equipo y el puntaje de cada jugador.
- **Recursividad:** El motor de turnos del combate está implementado mediante una función recursiva (ejecutar_turno_recursivo). Cada llamada representa un turno; el caso base se activa cuando uno de los dos entrenadores se queda sin personajes disponibles.
- **Programación orientada a eventos:** Tkinter opera sobre un bucle de eventos (mainloop). Para integrar la lógica recursiva de turnos con la interfaz gráfica sin bloquearla, se emplean callbacks y el método after() para programar acciones diferidas.

Marco Teórico

- **Inteligencia artificial simple:** El Hollow elige su acción (atacar o cambiar de personaje) de forma aleatoria mediante `random.choice`. Si solo tiene un personaje vivo, siempre ataca.

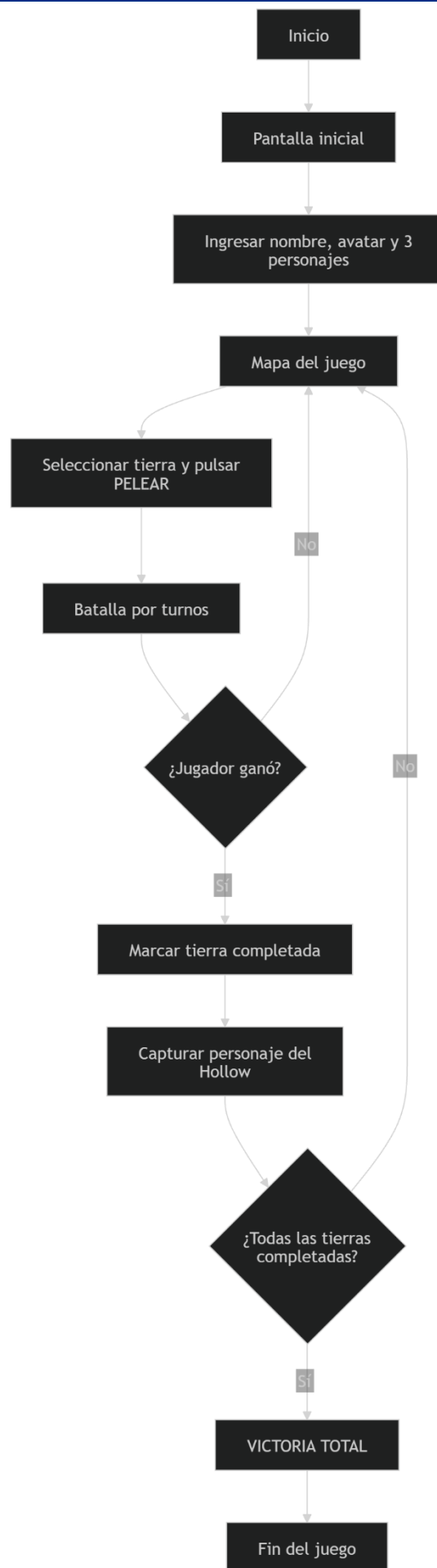
El programa está dividido en cinco módulos con responsabilidades claramente separadas, lo que facilita el mantenimiento y la comprensión del código.

Arquitectura de módulos:

main.py	Punto de entrada. Crea la ventana principal de Tkinter y lanza la pantalla inicial.
modelos.py	Define las clases Personaje y Entrenador, y la función cargar_personajes() que lee el archivo personajes.txt.
logica_batalla.py	Contiene el motor de turnos recursivo, el cálculo del turno de la IA y la lógica de captura de personajes.
pantalla_inicial.py	Pantalla de bienvenida donde el jugador ingresa su nombre, elige su avatar y selecciona exactamente 3 personajes.
pantalla_mapa.py	Muestra el mapa con las 5 tierras, el progreso del jugador y permite seleccionar la batalla siguiente.
pantalla_batalla.py	Renderiza la pantalla de combate con los paneles de ambos entrenadores, el log de batalla y los botones de acción.

Diagrama de flujo general del sistema:

El siguiente diagrama resume el flujo de ejecución principal del juego, desde el inicio hasta la condición de victoria o derrota:



Resultados Obtenidos

Descripción de las Funciones Principales

A continuación se describen las funciones más relevantes del sistema, incluyendo su módulo de origen, entradas, salidas y propósito:

■ cargar_personajes(ruta)

Módulo: modelos.py

Entrada: Ruta del archivo personajes.txt

Salida: Lista de objetos Personaje

Descripción: Lee el archivo de texto línea por línea y construye un objeto Personaje por cada registro con formato nombre, hp, atk, def. Los guiones bajos en el nombre se convierten en espacios (ej. 'Jose_David' → 'Jose David'). Es el punto de partida de toda la información del juego.

■ calcular_danio(defensor)

Módulo: modelos.py — clase Personaje

Entrada: Objeto Personaje defensor

Salida: Entero — daño causado (mínimo

1)

Descripción: Aplica la fórmula de daño: ATK del atacante menos DEF del defensor. Si el resultado es cero o negativo, el daño mínimo garantizado es 1, lo que asegura que toda batalla avanza sin quedar atascada.

■ ejecutar_turno_recursoivo(...)

Módulo: logica_batalla.py

Entrada: Entrenadores jugador y hollow, indicador de turno, callbacks de UI y fin

Salida: Ninguna (opera mediante callbacks)

Resultados Obtenidos

Descripción: Núcleo recursivo del sistema de combate. Cada llamada representa un turno. El caso base se activa cuando alguno de los dos entrenadores ya no tiene personajes con vida, momento en el que invoca `callback_fin`. El caso recursivo delega la acción a la interfaz gráfica mediante `callback_actualizar_ui` y se vuelve a llamar a sí misma con el turno alternado una vez que la acción fue procesada.

■ `_procesar_captura(turno_jugador, jugador, hollow)`

Módulo: `logica_batalla.py`

Entrada: Indicador de quién ganó el turno, ambos entrenadores

Salida: Ninguna (modifica los objetos en memoria)

Descripción: Cuando un personaje queda KO, el ganador lo captura. La función remueve al personaje derrotado del equipo del perdedor, lo añade al equipo del ganador y restaura su vida al máximo. Luego ajusta el índice de personaje activo del perdedor para apuntar al siguiente personaje con vida disponible.

■ **calcular_turno(atacante, defensor)**

Módulo: logica_batalla.py

Entrada: Dos objetos Entrenador

Salida: Diccionario con accion, danio, mensaje y ko

Descripción: Simula el turno de la IA del Hollow. Primero llama a `elegir_accion_random()` para decidir entre atacar o cambiar. Si cambia, busca el siguiente personaje vivo disponible. Si ataca, calcula el daño y determina si el defensor quedó KO. Devuelve un diccionario con toda la información necesaria para actualizar la UI.

■ **elegir_accion_random()**

Módulo: modelos.py — clase Entrenador

Entrada: Ninguna

Salida: String: 'atacar' o 'cambiar'

Descripción: IA simplificada del Hollow. Si el entrenador tiene más de un personaje vivo, elige aleatoriamente entre atacar o cambiar mediante `random.choice`. Si solo queda un personaje vivo, siempre ataca, ya que no tiene opciones de cambio.

■ **crear_pantalla_batalla(...)**

Módulo: pantalla_batalla.py

Entrada: Ventana, jugador, hollow, avatar, lista de personajes, completadas, índice de tierra

Salida: Ninguna (construye widgets en la ventana)

Descripción: Construye toda la pantalla de combate: los paneles visuales de ambos entrenadores con imagen, barra de HP y equipo; el log de batalla con historial de acciones; y los botones ATACAR y CAMBIAR. Al finalizar la construcción, inicia el

motor recursivo de turnos.

■ **callback_actualizar_ui(turno_jugador, continuar)**

Módulo: pantalla_batalla.py

Entrada: Indicador de turno activo y función para continuar la recursión

Salida: Ninguna (activa/desactiva botones según el turno)

Descripción: Puente entre la lógica recursiva y la interfaz gráfica. Si es turno del jugador, habilita los botones y asigna las funciones de ataque y cambio. Si es turno del Hollow, deshabilita los botones y programa la acción de la IA con un retardo de 800 ms mediante `ventana.after()` para simular tiempo de 'pensamiento'.

Descripción de la Interfaz Gráfica

La interfaz gráfica del juego consta de tres pantallas principales que se suceden de forma secuencial dentro de la misma ventana de 1200×800 píxeles, utilizando una paleta de colores oscura de estilo fantástico:

Pantalla inicial (pantalla_inicial.py):

Es la primera pantalla que ve el jugador. Contiene un campo para ingresar el nombre del Guardián, tres opciones de avatar mediante botones de radio (Guerrero, Defensor, Mago) y una tabla con los 15 personajes disponibles para seleccionar exactamente 3 mediante casillas de verificación. Se valida que el nombre no esté vacío y que se hayan seleccionado exactamente 3 personajes antes de avanzar.

Pantalla del mapa (pantalla_mapa.py):

Muestra las cinco tierras del Imaginario como botones interactivos. Al seleccionar una tierra se actualiza un panel de información que indica el nombre del Hollow correspondiente y si la tierra ya fue liberada. Las tierras completadas cambian de color a verde y muestran un ícono de verificación. Un panel inferior muestra el estado actual del jugador: nombre, avatar, puntaje y cantidad de personajes activos.

Pantalla de batalla (pantalla_batalla.py):

Es la pantalla más compleja. Muestra los paneles del jugador y del Hollow en lados opuestos, cada uno con la imagen del personaje activo, su nombre, una barra de HP dinámica que cambia de color según el porcentaje de vida restante, el puntaje y los íconos del equipo completo. En la parte inferior aparece un registro de batalla de texto desplazable y los botones ATACAR y CAMBIAR que se habilitan y deshabilitan según el turno activo.

Conclusiones y Recomendaciones

Conclusiones

- El uso de recursividad para el motor de turnos resultó una solución elegante y coherente con el paradigma de Python, permitiendo separar claramente la lógica de combate de la capa de presentación gráfica.
- La programación orientada a objetos facilitó la organización del código: las clases `Personaje` y `Entrenador` encapsulan su propia lógica y pueden extenderse fácilmente sin modificar otros módulos.
- La integración de Tkinter con una lógica recursiva requirió el uso de callbacks y del método `after()` para evitar bloquear el hilo principal de la interfaz, lo que representó uno de los mayores retos técnicos del proyecto.
- El sistema de captura de personajes añade profundidad estratégica: el jugador puede crecer su equipo a lo largo de las cinco batallas si juega bien, lo que hace que cada partida sea diferente.
- El proyecto demostró que Python, a pesar de ser un lenguaje de aprendizaje, permite construir aplicaciones interactivas completas con interfaz gráfica utilizando únicamente su biblioteca estándar.

Recomendaciones

- Incorporar efectos de sonido para los ataques, capturas y victoria, lo que mejoraría significativamente la experiencia del jugador sin grandes cambios estructurales.
- Implementar un sistema de guardado y cargado de partida mediante archivos JSON, de modo que el jugador pueda continuar su progreso en una sesión posterior.
- Agregar animaciones simples en la barra de HP (transición gradual al bajar) y un

Conclusiones y Recomendaciones

efecto visual al momento de la captura de personajes.

- Ampliar la IA del Hollow con una estrategia más inteligente, por ejemplo, priorizar el cambio cuando el personaje activo tiene baja vida o cuando el rival tiene una defensa muy alta.
- Considerar migrar la interfaz a una biblioteca más moderna como CustomTkinter para obtener un aspecto visual más contemporáneo manteniendo la compatibilidad con el código existente.

Bibliografía

1. Anthropic. (2025). *Claude* [Herramienta de inteligencia artificial]. <https://claude.ai>. Utilizada como apoyo para resolver dudas puntuales de sintaxis y revisar errores de lógica durante el desarrollo.
2. BitBoss. (2022). *Programación orientada a objetos (POO): Abstracción, encapsulamiento, herencia, polimorfismo* [Video]. YouTube. <https://youtu.be/SI7O81GMG2A>
3. Brottskar, M.. (2023). *Python GUI con Tkinter — Curso completo* [Video]. YouTube. <https://youtu.be/6frbSDzvFVQ>
4. Clark, A.. (2024). *Pillow (PIL Fork) documentation*. <https://pillow.readthedocs.io>
5. MoureDev. (2023). *Curso de Git y GitHub desde cero* [Video]. YouTube. <https://youtu.be/9d7lq4oMyPk>
6. OpenAI. (2025). *ChatGPT* [Herramienta de inteligencia artificial]. <https://chat.openai.com>. Consultada ocasionalmente para aclarar conceptos de programación orientada a objetos y el funcionamiento del bucle de eventos de Tkinter.
7. Python Software Foundation. (2024). *Python 3 documentation*. <https://docs.python.org>
8. Python Software Foundation. (2024). *tkinter — Python interface to Tcl/Tk*. <https://docs.python.org/3/library/tkinter.html>